



तत् त्वं पूषन् अपावृणु
केन्द्रीय विद्यालय संगठन

PM SHRI KENDRIYA
VIDYALAYA 14G.T.C
SUBATHU

Project title:
Medical Data
Management
and analysis

BY: SAUMYA KUMAR &
DEEPANSHU

CERTIFICATE

This is to certify that Saumya Kumar has successfully completed the project file on Medical Data Management and analysis under the guidance a supervision of Mr. Vipin Ramola

Subject Teacher

External Teacher

ACKNOWLEDGMENT

I am extremely thankful to my teacher Mr. Vipin Ramola, for constant support, Motivation and insights provided throughout the project MEDICAL DATA MANAGEMENT AND ANALYSIS.

I appreciate our principal, Mrs. Asha Chaudhari, for permitting access to the well-equipped lab and resources required for the project.

My special thanks to my team member for assisting me during the experiments and measurements .

Also, I would like to show my gratitude to my classmates for supporting me during experiments. *The encouragement from my teacher, principal and our friends were involved. I will always remain grateful for their support.*

CONTENTS

- 1. Introduction to Python and
MYSQL Connector*
- 3. SOFTWARE REQUIREMENTS*
- 4. HARDWARE REQUIREMENTS*
- 5. INTRODUCTION*
- 6. Introduction to Def Functions*
- 7. Flow Charts*
- 8. PYTHON CODE*
- 9. MySQL Query*
- 10. INPUT/OUTPUT INTERFACE*
- 11. CONCLUSION*
- 12. BIBLIOGRAPHY*

Declaration

1. Saumya Kumar(CLASS:XII-C)

Role : Coding , writing ,output

Responsibilities: Saumya Kumar was responsible for writing the code and generating the output and to write, classify about code for the project. This involved designing the algorithm, implementing the code, debugging, and ensuring that the final program ran smoothly and met all the specified requirements.

2. Deepanshu (CLASS XII-B)

Role: Editing

Responsibilities: Deepanshu took on the editing tasks. This involved reviewing the written content for clarity, coherence, and grammatical accuracy. They also ensured that the final document was polished, well-organized, and free from errors, enhancing the overall quality and professionalism of our project.

Introduction to Python & MYSQL

It is a programming language which is easy to learn, wide use, case sensitive, multi languages support

It also includes libraries: pandas, matplotlib, MySQL

Matplotlib: this type of library helps us to put all data into graphic format

Introduction to MYSQL

This type of library is used to connect MySQL databases with python data which helps us to store data in it, modify, and analyze data with it.

System Software Requirements

- 1. Python should be installed*
- 2. MySQL. Connector library should be installed*

Hardware Requirements

- *Processor: Intel Core i3 or AMD Ryzen3*
 - *RAM: 4 GB*
 - *Storage: 15GB sufficient*
 - *Network: Lan or Wi-Fi*
- *Display: 1280 x 720 resolution monitor*
- *In Add on: Standard keyboard and Mouse*

INTRODUCTION

This Python program is designed to simulate a comprehensive MEDICAL DATA MANAGEMENT AND ANALYSIS, providing a user-friendly interface for various operations with data some functions and data analysis, data management.

Introduction to Functions

This program contains small basic functions but plays a vital role in it. Some of them:

1. Create connection:

This function is used to connect MYSQL Connector with PYTHON which is important to save data for future purposes.

2. Insert patient:

This function helps us to add details to the database.

3. Fetch all patients:

This function is used to get all records from the database.

4. Update patient diagnosis:

This function updates the status or diagnosis of a patient.

5. Delete patient:

This function removes the record of a patient.

6. Analyze patient age:(Basic)

Calculates the average age of a patient

7. Analyze Diagnosis count:

This function gives the nod of patients suffering from a biological problem

8. Visualize age by diagnosis:

This function retrieves the average age of patients grouped by diagnosis and plots the results in a bar chart.

9. Visualize_age_by_gender:

It visualizes the average age grouped by gender in form of histogram.

10. Visualize_patients_by_treatment:

This function counts the number of patients receiving each treatment and visualizes the results in a pie chart.

11. MySQL connection close:

This function closes the connection of MySQL.

12. Main:

This function contains the full logic and user interface system which means get input and execute the function according to it.

13. login:

This function helps to make data private so that no one can access directly.

Medical Db project

loop

function

MYSQL
connection

Condition

loop

1. While
2. MYSQL connection
3. matplotlib

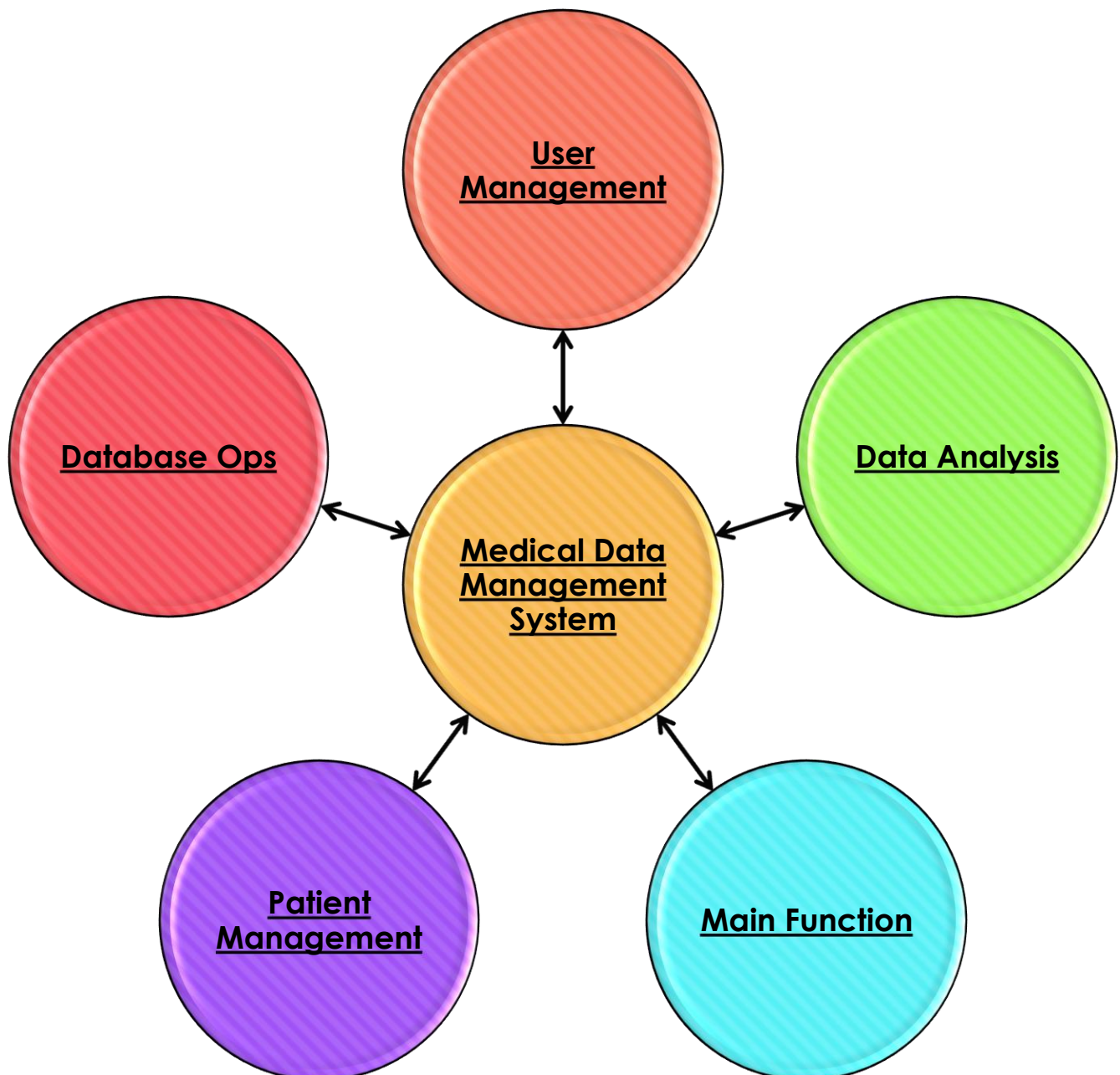
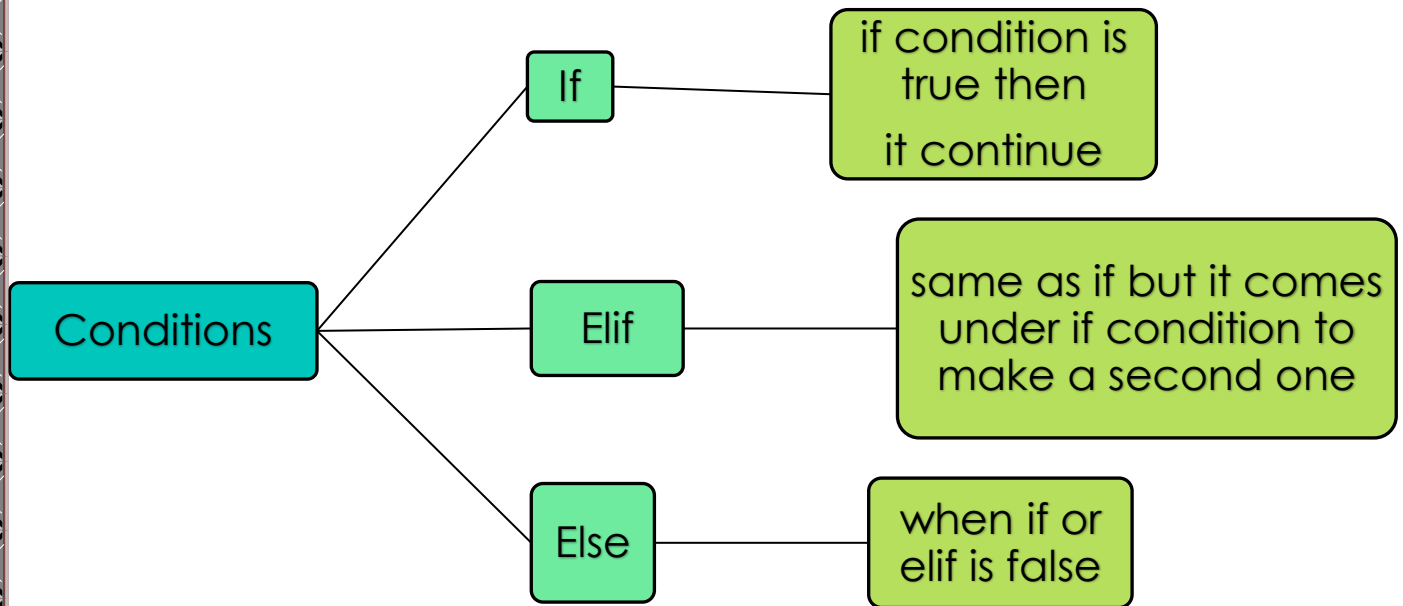
**MYSQL
Connection**

host=Localhost

user=root

password=tiger

database=MedicalDB



Let us begin the programming!

With Python

```
import mysql.connector as sql
from datetime import datetime
import matplotlib.pyplot as plt
```

```
# Login variables
```

```
n='Sam'
```

```
p="Sam@2024_25"
```

```
# Function to connect to the MySQL database
```

```
def create_db():
```

```
    db = sql.connect(
        host='localhost',
        database='MedicalDB',
        user='root',
        password='tiger'
```

```
)
```

```
    if db.is_connected():
```

```
        print("Connected to MySQL database")
    return db
```


Function to get user login credentials

def user():

name = input('Enter your name: ')

password = input('Enter your password: ')

return (name, password)

Function to insert a new patient

def insert_patient(db, name, age, gender, diagnosis, treatment):

cursor = db.cursor()

sql_query = """

INSERT INTO Patients (name, age, gender, diagnosis, treatment, admission_date)

VALUES (%s, %s, %s, %s, %s, %s)"""

admission_date = datetime.now().date()

values = (name, age, gender, diagnosis, treatment, admission_date)

cursor.execute(sql_query, values)

db.commit()

print(f"Patient {name} added successfully.")

Function to fetch all patients

def fetch_all_patients(db):

cursor = db.cursor()

*cursor.execute("SELECT * FROM Patients")*

rows = cursor.fetchall()

for row in rows:

print(row)

Function to update diagnosis for a specific patient

def update_patient_diagnosis(db, patient_id,

new_diagnosis):

cursor = db.cursor()

sql_query = "UPDATE Patients SET diagnosis =

%s WHERE patient_id = %s"

values = (new_diagnosis, patient_id)

cursor.execute(sql_query, values)

db.commit()

*print(f"Patient ID {patient_id}'s diagnosis updated
successfully.")*

Function to delete patient record

def delete_patient(db, patient_id):


```
cursor = db.cursor()
sql_query = "DELETE FROM Patients WHERE
patient_id = %s"
cursor.execute(sql_query, (patient_id,))
db.commit()
print(f"Patient ID {patient_id} deleted
successfully.")
```

```
# Function to analyze and print average age
def analyze_patient_age(db):
    cursor = db.cursor()
    cursor.execute("SELECT AVG(age) FROM
Patients")
    avg_age = cursor.fetchone()[0]
    print(f"The average age of patients is
{avg_age:.2f} years.")
```

```
# Function to analyze diagnosis count
def analyze_diagnosis_count(db):
    cursor = db.cursor()
    cursor.execute("SELECT diagnosis, COUNT(*)
FROM Patients GROUP BY diagnosis")
```

```
results = cursor.fetchall()
print("Diagnosis count:")
for diagnosis, count in results:
    print(f"{diagnosis}: {count} patients")
```

Function to visualize average age by diagnosis

```
def visualize_age_by_diagnosis(db):
```

```
    cursor = db.cursor()
```

```
    cursor.execute("SELECT diagnosis, AVG(age)
```

```
FROM Patients GROUP BY diagnosis")
```

```
    results = cursor.fetchall()
```

```
    diagnoses = [row[0] for row in results]
```

```
    avg_ages = [row[1] for row in results]
```

```
plt.figure(figsize=(10, 6))
```

```
plt.bar(diagnoses, avg_ages, color='skyblue')
```

```
plt.title('Average Age of Patients by Diagnosis')
```

```
plt.xlabel('Diagnosis')
```

```
plt.ylabel('Average Age')
```

```
plt.xticks(rotation=45)
```

```
plt.tight_layout()
```

```
plt.show()
```

```
# Function to visualize age distribution by gender
```

```
def visualize_age_by_gender(db):
```

```
    cursor = db.cursor()
```

```
    cursor.execute("SELECT gender, age FROM  
Patients")
```

```
    results = cursor.fetchall()
```

```
# Separate ages by gender
```

```
ages_by_gender = {}
```

```
for row in results:
```

```
    gender = row[0]
```

```
    age = row[1]
```

```
    if gender not in ages_by_gender:
```

```
        ages_by_gender[gender] = []
```

```
    ages_by_gender[gender].append(age)
```

```
# Create histogram
```

```
plt.figure(figsize=(10, 6))
```

```
for gender, ages in ages_by_gender.items():
```

```
plt.hist(ages, bins=10, alpha=0.5,  
label=gender)
```

```
plt.title('Age Distribution of Patients by Gender')  
plt.xlabel('Age')  
plt.ylabel('Frequency')  
plt.legend(title='Gender')  
plt.xticks(rotation=45)  
plt.tight_layout()  
plt.show()
```

```
# Function to visualize number of patients by  
treatment
```

```
def visualize_patients_by_treatment(db):  
    cursor = db.cursor()  
    cursor.execute("SELECT treatment, COUNT(*)  
FROM Patients GROUP BY treatment")  
    results = cursor.fetchall()  
  
    treatments = [row[0] for row in results]  
    counts = [row[1] for row in results]
```

```

plt.figure(figsize=(10, 6))
plt.pie(counts, labels=treatments,
autopct='%1.1f%%', colors=['lightgreen', 'lightcoral',
'lightskyblue'])
plt.title('Number of Patients by Treatment')
plt.axis('equal') # Equal aspect ratio ensures that
pie chart is a circle.
plt.tight_layout()
plt.show()

# Function to close MySQL db
def close_db(db):
    if db.is_connected():
        db.close()
        print("MySQL db is closed")

# Main function to run the system
def main():
    name, password = user()
    if n == name and p == password:
        print('Correct\n', 'Welcome Sir')
        while True:

```

```
db = create_db()
```

```
choice = int(input(""
```

```
MEDICAL DATA MANAGEMENT AND
```

```
ANALYSIS\n
```

```
1. Add New Patient\n
```

```
2. Show all records\n
```

```
3. Update diagnosis\n
```

```
4. Data Analysis on age\n
```

```
5. Data Analysis on diagnosis\n
```

```
6. Delete record of a patient\n
```

```
7. Visualize Average Age by Diagnosis\n
```

```
8. Visualize Average Age by Gender\n
```

```
9. Visualize Number of Patients by
```

```
Treatment\n
```

```
10. Exit\n
```

```
Enter your choice: """))
```

```
if choice == 1:
```

```
    name = input('Enter your name: ')
```

```
    age = int(input('Enter your age: '))
```

```
    gender = input('Enter your gender: ')
```

```
        problem = input('Enter your problem:
    ')

        treatment = input('Enter the treatment
which doctor starts: ')

        insert_patient(db, name, age, gender,
problem, treatment)

        print("\nAll Patient Data:")

    elif choice == 2:
        fetch_all_patients(db)
        print("\nData Analysis:")

    elif choice == 3:
        new_diagnosis = input("Enter the
new diagnosis: ")
        patient_id = int(input('Enter patient
ID: '))

        update_patient_diagnosis(db,
patient_id, new_diagnosis)

    elif choice == 4:
        print("\nData Analysis:")
```


analyze_patient_age(db)

elif choice == 5:

print("\nData Analysis:")

analyze_diagnosis_count(db)

elif choice == 6:

print("\nAll Patient Data:")

fetch_all_patients(db)

patient_id = int(input('Enter patient

ID: '))

delete_patient(db, patient_id)

elif choice == 7:

visualize_age_by_diagnosis(db)

elif choice == 8:

visualize_age_by_gender(db)

elif choice == 9:

visualize_patients_by_treatment(db)

```
        else:
            end = input('Do you want to exit (y,
n): ')

            if end == 'y':
                close_db(db)
                break

        else:
            print('Continue.....')

if __name__ == "__main__":
    main()
```

Let Us Have a Look on MySQL Query

– Active: 1728413654490@@127.0.0.1@3306

```
CREATE DATABASE MedicalDB;
```

```
USE MedicalDB;
```

```
CREATE TABLE Patients (  
    patient_id INT PRIMARY KEY  
    AUTO_INCREMENT,  
    name VARCHAR(50),  
    age INT,  
    gender ENUM('Male', 'Female', 'Other'),  
    diagnosis VARCHAR(100),  
    treatment VARCHAR(100),  
    admission date DATE );
```

Let us insert data in it before executing

```
-- Active: 1728413654490@@@127.0.0.1@3306
```

```
USE MedicalDB;
```

```
Select*from patients;
```

```
INSERT INTO patients (patient_id, name, age,  
gender, diagnosis, treatment, admission date)
```

```
VALUES
```

```
    (1, 'Ram', 20, 'Male', 'Mental health', 'Take care  
of your physical health', '2024-06-06'),
```

```
    (2, 'Mohammed', 50, 'Male', 'cardiovascular  
disease', 'Control blood pressure', '2024-06-07'),
```

```
    (3, 'Sunita', 45, 'Female', 'Lower respiratory  
disease', 'drinking plenty of fluids', '2024-06-08'),
```

```
    (4, 'Sri', 65, 'Female', 'heart disease', 'Statins',  
'2024-06-09'),
```

```
    (5, 'Sumit', 30, 'Male', 'Diabetes', 'Control sugar  
level', '2024-06-10');
```

Output of MySQL

From Start:

```
Enter password: *****
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 11
Server version: 8.0.40 MySQL Community Server - GPL

Copyright (c) 2000, 2024, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> CREATE DATABASE MedicalDB;
Query OK, 1 row affected (0.07 sec)
```

Table and Table structure

```
mysql> USE MedicalDB;
Database changed
mysql> CREATE TABLE Patients (
  ->     patient_id INT PRIMARY KEY AUTO_INCREMENT,
  ->
  ->     name VARCHAR(50),
  ->
  ->     age INT,
  ->
  ->     gender ENUM('Male', 'Female', 'Other'),
  ->
  ->     diagnosis VARCHAR(100),
  ->
  ->     treatment VARCHAR(100),
  ->     admission_date DATE
  -> );
Query OK, 0 rows affected (0.15 sec)
```

Table structure

```
mysql> desc patients;
```

Field	Type	Null	Key	Default	Extra
patient_id	int	NO	PRI	NULL	auto_increment
name	varchar(50)	YES		NULL	
age	int	YES		NULL	
gender	enum('Male','Female','Other')	YES		NULL	
diagnosis	varchar(100)	YES		NULL	
treatment	varchar(100)	YES		NULL	
admission_date	date	YES		NULL	

7 rows in set (0.03 sec)

Data Entry

```
mysql> INSERT INTO patients (patient_id, name, age, gender, diagnosis, treatment, admission_date) VALUES
S
-> (1, 'Ram', 20, 'Male', 'Mental health', 'Take care of your physical health', '2024-06-06'),
-> (2, 'Mohammed', 50, 'Male', 'Cardiovascular disease', 'Control blood pressure', '2024-06-07'
),
-> (3, 'Sunita', 45, 'Female', 'Lower respiratory disease', 'drinking plenty of fluids', '2024-
06-08'),
-> (4, 'Sri', 65, 'Female', 'Heart Disease', 'Statins', '2024-06-09'),
-> (5, 'Sumit', 30, 'Male', 'Diabetes', 'Control sugar level', '2024-06-10');
Query OK, 5 rows affected (0.01 sec)
Records: 5 Duplicates: 0 Warnings: 0
```

Data in table before python code run

```
mysql> Select*from patients;
```

patient_id	name	age	gender	diagnosis	treatment
admission_date					
1	Ram	20	Male	Mental health	Take care of your physical health
2024-06-06					
2	Mohammed	50	Male	Cardiovascular disease	Control blood pressure
2024-06-07					
3	Sunita	45	Female	Lower respiratory disease	drinking plenty of fluids
2024-06-08					
4	Sri	65	Female	Heart Disease	Statins
2024-06-09					
5	Sumit	30	Male	Diabetes	Control sugar level
2024-06-10					

5 rows in set (0.00 sec)

Python Input and Output

Login

```
PS D:\> & C:/Users/dell/AppData/Local/Microsoft/WindowsApps/python3.12.exe d:/sam.py
enter your name:Sam
enter your password:Sam@2024_25
correct
  wellcome Sir
Connected to MySQL database
```

Main:

MEDICAL DATA MANAGEMENT AND ANALYSIS

1. Add New Patient
2. Show all records
3. Update diagnosis
4. Data Analysis on age
5. Data Analysis on diagnosis
6. Delete record of a patient
7. Visualize Average Age by Diagnosis
8. Visualize Average Age by Gender
9. Visualize Number of Patients by Treatment
10. Exit

Enter your choice:

If 1:

```
Enter your name: Saumya
Enter your age: 17
Enter your gender: Male
Enter your problem: Body Pain
Enter the treatment which doctor starts: Yoga
Patient Saumya added successfully.
```

Elif 2:

```
(1, 'Ram', 20, 'Male', 'Over Weight', 'Take care of your physical health', datetime.date(2024, 6, 6))
(2, 'Mohammed', 50, 'Male', 'Cardiovascular disease', 'Control blood pressure', datetime.date(2024, 6, 7))
(3, 'Sunita', 45, 'Female', 'Lower respiratory disease', 'drinking plenty of fluids', datetime.date(2024, 6, 8))
(4, 'Sri', 65, 'Female', 'Heart Disease', 'Statins', datetime.date(2024, 6, 9))
(7, 'Deepanshu', 17, 'Male', 'Body pain', 'Yoga', datetime.date(2024, 11, 7))
(8, 'Saumya', 17, 'Male', 'Body Pain', 'Yoga', datetime.date(2024, 11, 9))
```

Elif 3:

```
Enter the new diagnosis: stress
Enter patient ID: 8
Patient ID 8's diagnosis updated successfully.
```

Elif 4:

```
Data Analysis:
The average age of patients is 35.67 years.
```

```
Data Analysis:
Diagnosis count:
Over Weight: 1 patients
Cardiovascular disease: 1 patients
Lower respiratory disease: 1 patients
Heart Disease: 1 patients
Body pain: 1 patients
stress: 1 patients
```

Elif 5:

Elif 6:

All Patient Data:

```
(1, 'Ram', 20, 'Male', 'Over Weight', 'Take care of your physical health', datetime.date(2024, 6, 6))
(2, 'Mohammed', 50, 'Male', 'Cardiovascular disease', 'Control blood pressure', datetime.date(2024, 6, 7))
(3, 'Sunita', 45, 'Female', 'Lower respiratory disease', 'drinking plenty of fluids', datetime.date(2024, 6, 8))
(4, 'Sri', 65, 'Female', 'Heart Disease', 'Statins', datetime.date(2024, 6, 9))
(7, 'Deepanshu', 17, 'Male', 'Body pain', 'Yoga', datetime.date(2024, 11, 7))
(8, 'Saumya', 17, 'Male', 'stress', 'Yoga', datetime.date(2024, 11, 9))
```

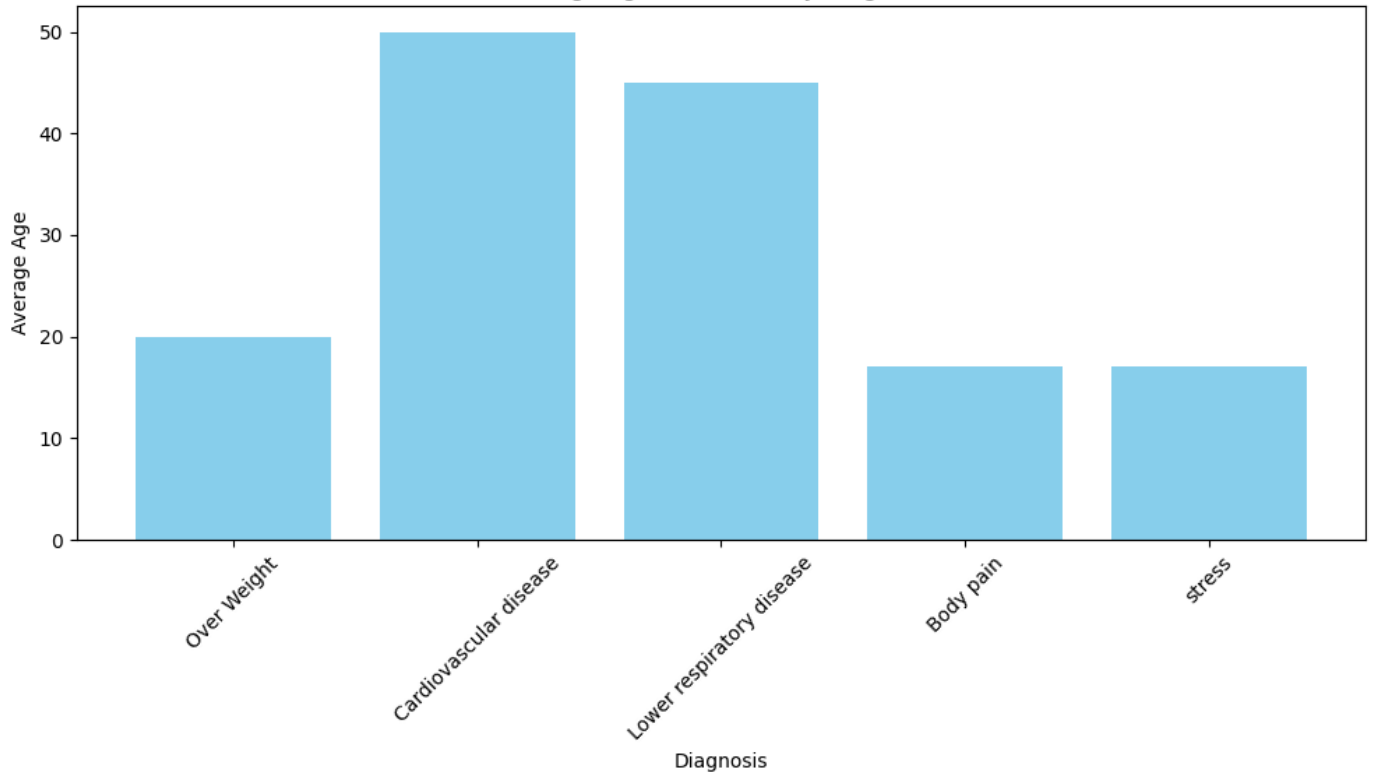
Enter patient ID: 4

Patient ID 4 deleted successfully.

Elif 7:

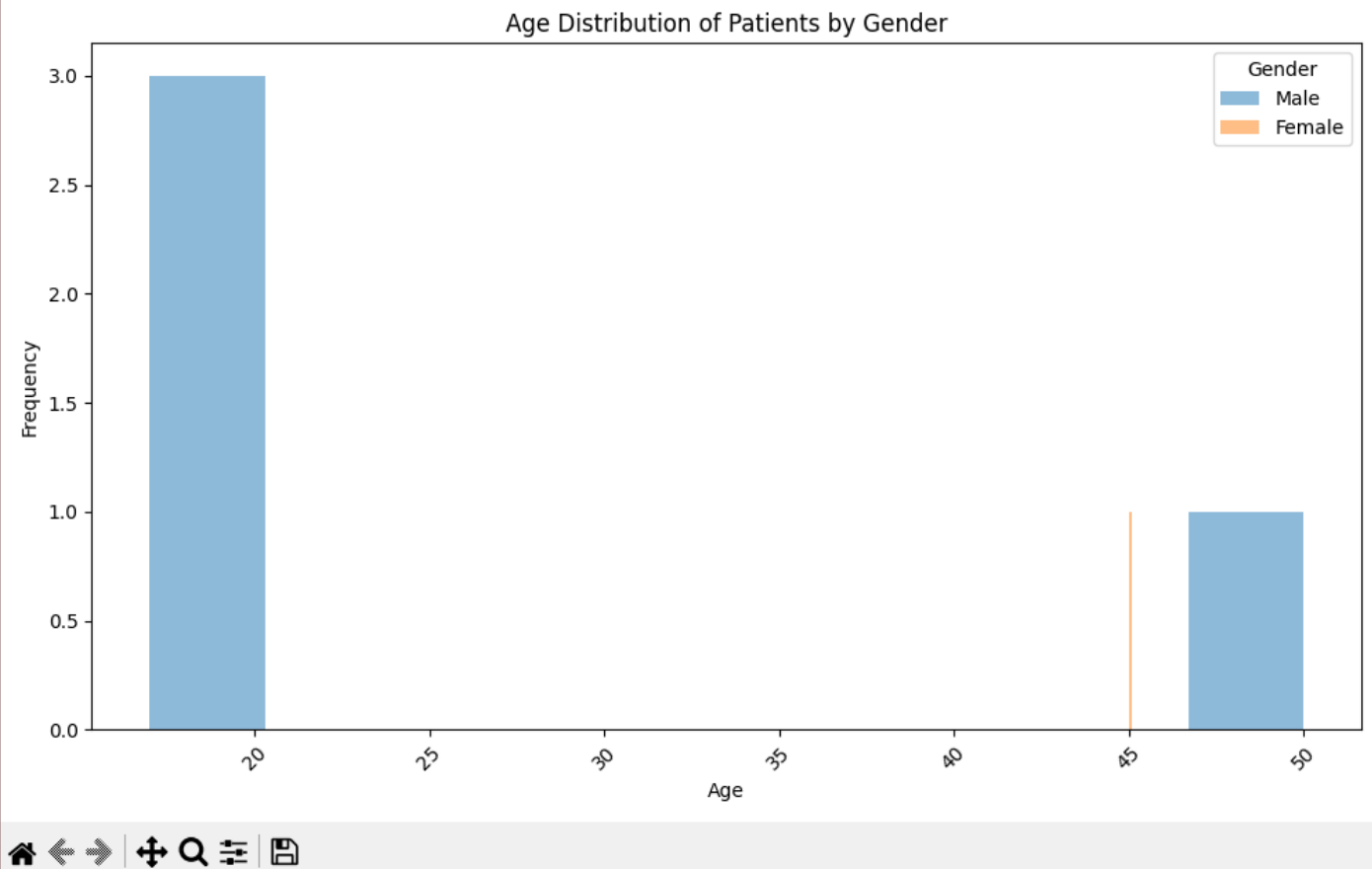
Figure 1

Average Age of Patients by Diagnosis



Elif 8:

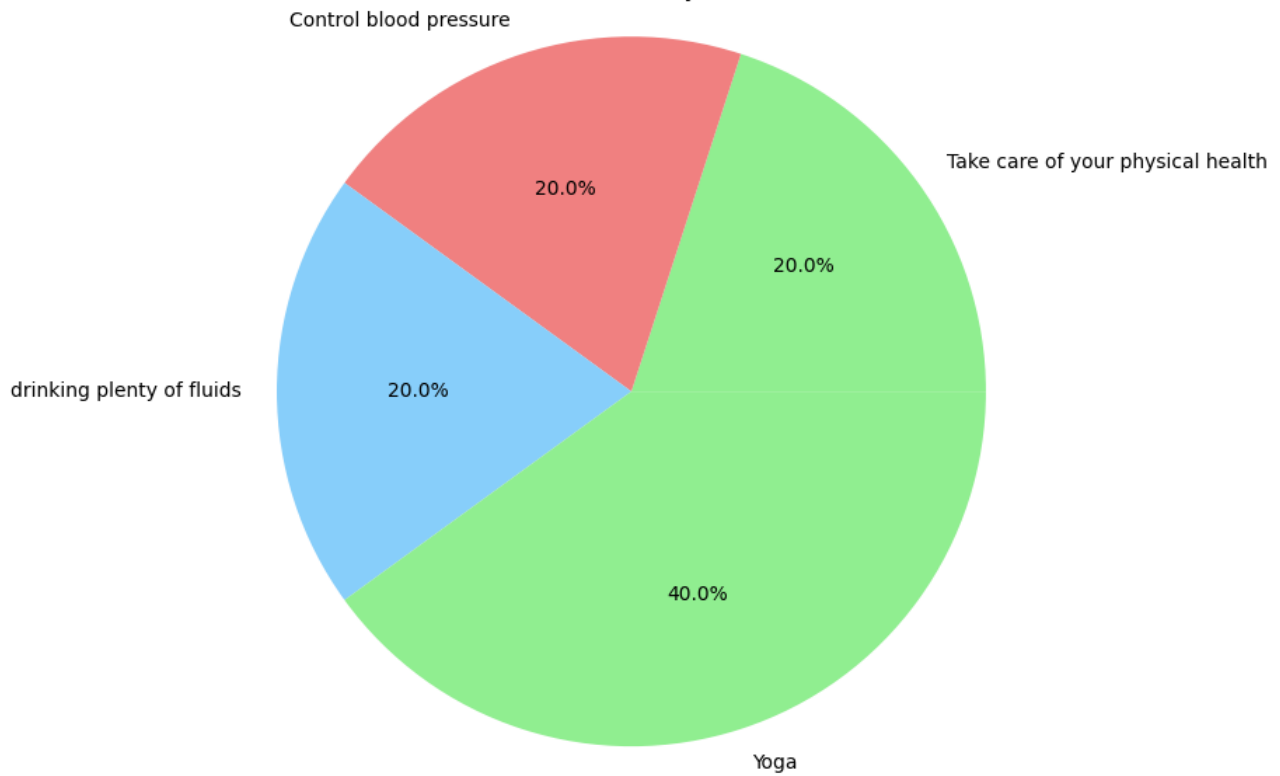
Figure 1



Elif 9:

Figure 1

Number of Patients by Treatment



Elif 10:

```
Do you want to exit (y, n): y
MySQL db is closed
```

CONCLUSION

The Medical Data Management Project successfully demonstrates a robust system for managing and analyzing patient data. Through its user-friendly interface, it allows healthcare professionals to efficiently perform essential tasks, such as adding new patients, updating diagnoses, and conducting data analyses based on age and diagnosis.

The implementation of password-protected access ensures that sensitive medical information is safeguarded, thus enhancing data privacy and security. The functionality to delete patient records further emphasizes the system's management capabilities, providing flexibility in handling patient information.

Bibliography

- 1. NCERT Books*
- 2. Google Gemini*
- 3. Sumit Arora 12 IP book*
- 4. Articles*
- 5. Web browser*